

Context Compression Engine

Backend Build Guide — Revised (Zero-Cost Architecture) · v2.0

What changed from v1.0 and why

v1.0 assumed Claude API for every LLM call. Claude has no permanent free API tier (only a one-time ~\$5 signup credit), so running the full pipeline on Claude during dev and testing would burn that credit before demo day. v2.0 splits the work across two free-tier providers based on call volume, so the whole pipeline runs at \$0.

Pipeline stage	v1.0 provider	v2.0 provider	Why
Stage 1 — structural compression	None (pure JS)	Unchanged	Already free, no LLM needed
Stage 2 — chunk scoring + compression	Claude API	Groq (Llama 3.3 70B)	High call volume — every chunk, every test run. Groq's free tier is sufficient.
Stage 4 — accuracy validation (QA API judge)	Claude API	Claude Haiku 4.5 (kept)	Low call volume — a handful of calls per run. The \$5 free tier covers this.
Fallback if Stage 4 credit runs out	None	Gemini 2.5 Flash (free, permanent)	Zero-cost backup so demo never blocks on a depleted credit.

API contracts (field names in /api/compress, /api/validate, /api/history) are unchanged from the SRS — this is purely a provider swap behind the same JSON shape, so Frontend and Database work is unaffected.

1. Your responsibilities (unchanged)

- Build the two API routes: `/api/compress` and `/api/validate`, plus `/api/history`
- Implement Stage 1 (structural) compression logic
- Orchestrate Stage 2 (semantic) compression via Groq API calls
- Implement the accuracy-validation Q&A; + similarity scoring logic via Claude API
- Handle token counting, cost estimation, latency measurement
- Connect to the database (write run logs, read history)
- Handle errors, timeouts, and free-tier rate limits gracefully with fallbacks

2. Tech stack

- Node.js via Next.js API routes (`app/api/.../route.ts`)
- **groq-sdk** (OpenAI-compatible client) for Stage 2 chunk scoring + compression
- **@anthropic-ai/sdk** for Stage 4 validation Q&A; + similarity judge (Claude Haiku 4.5)
- **@google/generative-ai** as a documented fallback client (Gemini 2.5 Flash) if the Claude credit is exhausted
- `js-tiktoken` (`cl100k_base`) for a single consistent approximate token counter across both providers
- `@supabase/supabase-js` for database calls

3. Project setup (30 min)

- Environment variables: `GROQ_API_KEY`, `ANTHROPIC_API_KEY`, `GEMINI_API_KEY` (fallback), `SUPABASE_URL`, `SUPABASE_SERVICE_KEY`
- Install: `npm install groq-sdk @anthropic-ai/sdk @google/generative-ai @supabase/supabase-js js-tiktoken`

```
/app/api/compress/route.ts
/app/api/validate/route.ts
/app/api/history/route.ts
/lib/compression/stage1.ts
/lib/compression/stage2.ts      (now calls Groq)
/lib/compression/chunker.ts
/lib/validation/qa.ts          (now calls Claude, falls back to Gemini)
/lib/validation/similarity.ts  (Claude judge, falls back to Gemini)
/lib/tokenCount.ts
/lib/groqClient.ts             (new)
/lib/claudeClient.ts
/lib/geminiClient.ts           (new, fallback only)
/lib/supabaseClient.ts
/lib/rateLimiter.ts            (new – see section 6)
```

4. Utility setup (30 min)

- `lib/groqClient.ts` — configured Groq client, reads `GROQ_API_KEY` from env, uses the OpenAI-compatible `chat.completions.create()` interface
- `lib/claudeClient.ts` — configured Anthropic client (unchanged from v1.0), model set to `claude-haiku-4-5`

- lib/geminiClient.ts — configured Gemini client, only invoked as a fallback (see section 6)
- lib/tokenCount.ts — one countTokens(text) function used everywhere, so ratios stay comparable across providers

5. Stage 1 — structural compression (1.5–2 hrs) — unchanged from v1.0

- removeExactDuplicateLines(text)
- collapseWhitespace(text)
- stripRepeatedBoilerplate(text) — mark collapsed blocks, e.g. [boilerplate collapsed x12]
- collapseNearDuplicateLines(text) — Levenshtein/Jaccard similarity, threshold ~85%
- Combine into runStage1(text): { compressedText, originalTokens, stage1Tokens }

6. Stage 2 — semantic compression via Groq (3–4 hrs)

Same chunking + scoring + compression design as v1.0, redirected to Groq's Llama 3.3 70B. Two things change in practice:

- Groq's free tier caps around 30 requests/min and 6,000 tokens/min — batch chunks aggressively (10–15 per call) and add a queue/backoff so a burst of test runs doesn't 429
- Llama models are occasionally less strict about JSON-only output than Claude — keep the "return ONLY valid JSON" instruction and add a lightweight JSON-repair step (strip markdown fences, retry once) before falling back to the score-5 default

```
// lib/compression/stage2.ts (skeleton)
import { groq } from '../groqClient';

export async function scoreChunks(chunks: Chunk[]): Promise {
  const res = await groq.chat.completions.create({
    model: "llama-3.3-70b-versatile",
    temperature: 0,
    messages: [{ role: "user", content: buildScoringPrompt(chunks) }],
  });
  return parseScoresWithFallback(res.choices[0].message.content, chunks);
}

export async function compressLowScoreChunks(scored: ScoredChunk[]) {
  const low = scored.filter(c => c.score < 6);
  // batch multiple low-score chunks into one call where possible
  const res = await groq.chat.completions.create({
    model: "llama-3.3-70b-versatile",
    temperature: 0.2,
    messages: [{ role: "user", content: buildCompressionPrompt(low) }],
  });
  return mergeCompressed(res, scored);
}
```

7. /api/compress route (1 hr) — unchanged contract

```
POST /api/compress
  in: { originalText, options }
  out: { compressedText, stage1Text, originalTokens, stage1Tokens,
        finalTokens, compressionRatio, warning? }
```

- Validate input (non-empty, under ~50,000 chars)
- Run Stage 1 -> Stage 2 (Groq) -> compute compressionRatio
- On Groq failure/timeout/429: fall back to Stage-1-only result, set warning field

8. Stage 4 — /api/validate route via Claude, with Gemini fallback (1.5 hrs)

```
POST /api/validate
  in: { originalText, compressedText, question }
  out: { answerFull, answerCompressed, similarityScore,
        latencyFull, latencyCompressed, providerUsed }
```

- askClaude(context, question) — single function, measures latency, model = claude-haiku-4-5
- If Claude call fails (rate limit, credit exhausted, timeout) -> askGemini(context, question) as drop-in replacement, same return shape
- scoreSimilarity(answerA, answerB) — Claude-as-judge (0-100 + reasoning); falls back to Gemini judge on the same failure conditions
- Response includes providerUsed so the frontend/judges can see which model actually answered — good transparency for the demo

9. Database logging (1 hr) — unchanged from v1.0

- Write a row to runs after /api/compress and /api/validate both complete
- Add a providerUsed column (or reuse status) so you can show in the pitch which provider handled each run
- /api/history — GET, last 20 runs, ordered by created_at desc

10. Rate limits & fallbacks (new section — budget 1 hr)

- lib/rateLimiter.ts — simple token-bucket or queue in front of the Groq client so parallel chunk-scoring calls don't burst past 30 RPM during heavy testing
- Every external call (Groq, Claude, Gemini) wrapped with a 20s timeout and a single retry before falling back
- Fallback chain for Stage 2: Groq -> (if rate-limited) queue and retry once -> (if still failing) Stage-1-only result
- Fallback chain for Stage 4: Claude -> Gemini -> partial result with error flag, never crash the response
- Log every fallback trigger to console with enough context to debug quickly during the live demo

What "done" looks like

- /api/compress reliably takes raw text and returns compressed text + accurate metrics in under 20 seconds for a 5,000-token input, using Groq for Stage 2

- /api/validate reliably returns two answers + a similarity score, using Claude (or Gemini on fallback)
- Both routes handle bad/empty input and free-tier rate limits without crashing
- History is retrievable via /api/history
- Total spend across the whole build + demo: \$0

Coordination notes

- JSON field names are unchanged from SRS v1.0 — no need to re-sync with Frontend or Database teams
- Tell the team lead about the Claude -> Groq swap for Stage 2 before finalizing, in case "Claude only" was an eligibility requirement rather than a technical preference
- Give Frontend a stub response early (Day 1, Hour 1) matching the contract above, exactly as in v1.0